SOC Threat Hunting and Investigation Engine

Technical Documentation

Project type: Defensive security / SOC investigation support **Core technologies:** Python, DuckDB, CSV, JSONL, structured and semi-structured security logs **Document purpose:** Explain the implementation, operation, validation approach, investigation checks, outputs, limitations, and reproducibility of the project.

1. Project Overview

The SOC Threat Hunting and Investigation Engine is a local data-processing and investigation-support project. It ingests security event data from multiple sources, normalizes the records into source-specific DuckDB tables, performs data-quality reviews, applies rule-based investigation checks, and produces investigation findings for analyst review.

The project is designed to make large log collections easier to inspect consistently. It preserves source context and records why an event was flagged. It does not independently determine that an attack succeeded, that a host is compromised, or that an alert represents a confirmed incident.

1.1 Objectives

1. Ingest DNS, application-security, firewall, Linux syslog, and Windows Security event data.
1. Normalize common event fields while retaining source-specific attributes.
1. Preserve source line references and raw event content where available.
1. Review data quality, metadata, identifiers, and duplicate conditions.
1. Apply source-specific investigation checks.
1. Generate a unified CSV investigation register and a compact summary.
1. Keep investigation outputs explainable and suitable for analyst review.

1.2 Scope

The current implementation performs source-by-source checks. Cross-source correlation is explicitly out of scope in the current register. The project is an investigation-support and triage tool, not an automated incident verdict engine or a replacement for a SIEM, EDR, firewall, or analyst-led incident response process.

1.3 Key implementation results

The current DuckDB database contains five normalized tables with **2,472,228 rows** in total:

Source	Table	Rows
DNS	`normalized_events`	350,000
Application security	`normalized_app_events`	438,739
Firewall	`normalized_firewall_events`	394,170
Linux syslog	`normalized_linux_events`	977,509
Windows Security	`normalized_windows_events`	311,810
Total		**2,472,228**

The unified investigation register contains **767,672 findings**:

Source	Findings
DNS	65,825
Application security	159,792
Firewall	275,111
Linux syslog	218,002
Windows Security	48,942

Source	Findings
Total	**767,672**

These are finding rows, not necessarily unique attacks, incidents, or compromised systems.

2. Architecture and Data Flow

The project uses a local, file-based workflow. Security logs are read from the project `logs` directory. Python ingestion code parses and normalizes the records, then writes normalized tables to a persistent DuckDB database. Validation and investigation scripts read the database and produce review artifacts.

```
Source log files
  |-- DNS JSONL
  |-- Application security log
  |-- Firewall log
  |-- Linux syslog
  `-- Windows Security event log
         |
         v
   Python ingestion layer
         |
         v
   Normalization and field extraction
         |
         v
   Persistent DuckDB database
         |
         +--> Database / source validation
         |
         +--> Source-specific investigation checks
         |
         +--> Windows validation findings CSV
         |
         v
   Unified investigation register
         |
         +--> investigation_register.csv
         `--> investigation_register_summary.txt
```

2.1 Main components

1. **Input layer:** Raw source logs stored under `logs`.
1. **Ingestion layer:** `ingestion.py` reads source files, parses records, extracts common and source-specific fields, and writes normalized tables.
1. **Storage layer:** `threat_hunt.duckdb` stores the normalized tables.
1. **Validation layer:** Supporting scripts inspect table integrity, duplicate identifiers, source metadata, source quality, and Windows event fields.
1. **Investigation layer:** `investigation_checks.py` creates a timestamped text report. `investigation_register.py` creates the unified CSV register and summary.
1. **Windows validation output:** `validate_windows_events.py` creates `windows_validation_findings.csv`, which is consumed by the unified register.

2.2 Separation of outputs

The timestamped investigation report and unified investigation register are separate workflows:

1. `investigation_checks.py` writes a timestamped text report.
1. `investigation_register.py` writes a structured CSV register and a summary text file.
1. Windows validation is run separately and its CSV findings are imported by the register generator.

The outputs should not be treated as interchangeable.

3. Environment and Dependencies

The development environment used for the current implementation was:

1. Windows 11
1. Python 3.14.6
1. DuckDB 1.5.4

The Python package dependency identified in the project is DuckDB. Other imports used by the scripts are from the Python standard library.

A reproducible dependency file should pin the tested DuckDB version:

```
duckdb==1.5.4
```

3.1 Verify the environment

From the project root:

```
py --version
py -m pip show duckdb
```

Install dependencies:

```
py -m pip install -r requirements.txt
```

The exact Python version available on another machine may differ. Use a Python release compatible with the pinned DuckDB package and verify the scripts before processing the full datasets.

4. Project Structure

The working project uses the following layout:

```
splunk-threat-hunt-engine/
|
|-- logs/
|   |-- dns_query_logs.jsonl
|   |-- app_sec_audit.log
|   |-- firewall_traffic.log
|   |-- linux_syslog.log
|   `-- windows_security_events.log
|
|-- reports/
|   |-- investigation_register.csv
|   |-- investigation_register_summary.txt
|   `-- archive/
|       `-- investigation_report_*.txt
|
|-- src/
|   `-- threat_hunt_engine/
|       |-- __init__.py
|       |-- ingestion.py
|       |-- investigation_checks.py
|       |-- investigation_register.py
|       |-- database_validation.py
|       |-- duplicate_id_check.py
|       |-- source_metadata_review.py
|       |-- source_quality_review.py
|       |-- validate_windows_events.py
|       `-- windows_validation_findings.csv
|
|-- docs/
|   `-- SOC_Threat_Hunting_Engine_Technical_Documentation.pdf
|
|-- threat_hunt.duckdb
|-- requirements.txt
|-- README.md
|-- .gitignore
`-- LICENSE
```

This describes the working layout. A public repository should not automatically include every working data artifact. In particular, the raw logs, DuckDB database, large generated reports, and validation CSV require a separate redistribution and privacy review before publication.

5. Ingestion and Normalization

5.1 Ingestion entry point

Run the full ingestion workflow from the project root:

```
py -m src.threat_hunt_engine.ingestion
```

The ingestion script derives the project root from its own file location and uses project-relative paths. The current path pattern is:

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
LOG_DIR = PROJECT_ROOT / "logs"
DB_FILE = PROJECT_ROOT / "threat_hunt.duckdb"
```

This avoids dependence on a hard-coded machine-specific directory.

The full ingestion workflow expects all five source log files to be present. It rebuilds the normalized source tables using `CREATE OR REPLACE TABLE`. Running it again therefore replaces the existing normalized tables rather than appending a second copy of the same input.

5.2 Linux-only ingestion

The Linux-only option is:

```
py -m src.threat_hunt_engine.ingestion --linux-only
```

This mode rebuilds `normalized_linux_events` without rebuilding the other source tables. It requires the Linux source file and should be used in the context of the existing project database when the intention is to preserve the other normalized tables.

5.3 Normalized table inventory

The current schema contains the following tables:

Table	Rows	Source-specific fields
`normalized_events`	350,000	DNS query, response, resolver, entropy, query length and labels
`normalized_app_events`	438,739	Application service, method, URI, status, bytes, latency, user agent, authentication and attack classification
`normalized_firewall_events`	394,170	Action, protocol, source/destination ports, rule, TCP flags, bytes, packets, direction and interface
`normalized_linux_events`	977,509	Linux hostname, process, PID, message, authentication result and facility
`normalized_windows_events`	311,810	Windows event ID, process, command line, recorded label, channel, ports and event type

5.4 Common fields

The tables share a common event structure where applicable:

1. `event_id`: internal event identifier.
1. `source_event_id`: source-provided identifier, where available.
1. `source_line_number`: original source line reference.
1. `source_type` and `source_file`: source identification.
1. `event_category`, `event_action`, and `severity`: event classification fields.
1. `source_timestamp`: timestamp text from the source.
1. `event_timestamp`: normalized DuckDB timestamp, where parsing is supported.
1. `src_ip`, `dst_ip`, `host`, and `username`: common context fields.
1. `source_labels`: source-provided or extracted labels.
1. `raw_event`: original or retained raw event representation, depending on the source.

The schema is deliberately broad enough to support common investigation fields while retaining source-specific columns.

5.5 Raw event storage differences

The raw event column is not the same DuckDB type in every table:

1. `normalized_events.raw_event` is `JSON`.
1. `normalized_app_events.raw_event`, `normalized_firewall_events.raw_event`, `normalized_linux_events.raw_event`, and `normalized_windows_events.raw_event` are `VARCHAR`.

Consumers should inspect the table schema rather than assume that every raw event column has the same type.

6. Data Quality and Validation

Validation is treated as a separate activity from threat classification. A malformed or unusual record is not automatically malicious, and a valid record is not automatically benign.

The project includes these supporting scripts:

Script	Purpose
`database_validation.py`	Read-only database and table validation
`duplicate_id_check.py`	Checks internal/source identifier duplication conditions
`source_metadata_review.py`	Reviews source-file metadata and source/database relationships
`source_quality_review.py`	Reviews source quality and normalized data conditions
`validate_windows_events.py`	Applies Windows-specific event validation and writes a findings CSV

Run the modules from the project root:

```
py -m src.threat_hunt_engine.database_validation
py -m src.threat_hunt_engine.duplicate_id_check
py -m src.threat_hunt_engine.source_metadata_review
py -m src.threat_hunt_engine.source_quality_review
py -m src.threat_hunt_engine.validate_windows_events
```

The database validation, metadata review, source-quality review, and Windows validation scripts connect to DuckDB in read-only mode. The Windows validator writes its findings to a separate CSV file; it does not modify the normalized Windows table.

6.1 Linux timestamp limitation

The Linux source timestamps do not include a year. The current normalized table retains the original timestamp text in `source_timestamp`, but the normalized `event_timestamp` field is NULL for all 977,509 Linux records. The validation review identified 977,509 unnormalized timestamps.

This is a known limitation of the current parser and source format. Do not use the missing normalized timestamp as evidence that an event did not occur. Timeline work involving Linux events must account for the original timestamp format and the missing year.

6.2 Linux source IP extraction

The Linux normalization logic was updated to selectively extract IPv4 addresses from common message patterns, including:

1. SSH `from` values.
1. PAM `rhost=` values.
1. UFW `SRC=` and `DST=` values.
1. SSH connection-closed and disconnect messages.

The Linux-only ingestion run processed 977,509 lines in approximately 11.08 seconds, or about 88,224 records per second, in the recorded development environment.

The validation review reported:

1. 175,285 records with `src_ip` populated.
1. 2,475 records with `dst_ip` populated.

1. 2,475 records with both fields populated.

These counts are specific to the reviewed dataset and current extraction logic; they are not a guarantee that every possible Linux message format is covered.

6.3 Identifier and preservation checks

The Linux-only validation reported:

1. 977,509 normalized rows.
1. 0 missing internal event IDs.
1. 0 duplicate internal IDs.
1. 0 repeated source-ID groups.
1. 0 excess source-ID occurrences.
1. 0 missing raw events.

The investigation workflow is designed to retain records for review rather than silently discard events simply because they are unusual or duplicated. Any deduplication in the register is performed using the investigation identifier and should not be confused with deleting source events from the database.

7. Source-Specific Investigation Checks

The checks below create review leads. Their severity values are triage priorities, not confirmed incident classifications.

7.1 DNS

Check	Findings	Severity
`DNS_SOURCE_LABEL_REVIEW`	45,921	MEDIUM
`DNS_HIGH_ENTROPY_QUERY_REVIEW`	12,010	MEDIUM
`DNS_LONG_QUERY_REVIEW`	7,894	LOW
Total	**65,825**	

Source-label review: Reviews source labels such as `suspicious-tld`, `dga`, `c2`, `beacon`, `high-entropy`, and `dns-tunneling`. These labels represent classifications or indicators in the source data. They do not independently prove infection, command-and-control communication, or data exfiltration.

High-entropy query review: Flags DNS queries where `dns_subdomain_entropy >= 4.5`. Reviewed examples included long, random-looking subdomains and entropy values around 5.0. Such patterns can be consistent with encoded DNS exchanges, but entropy alone does not prove tunnelling or exfiltration.

Long-query review: Flags DNS queries where `dns_query_length >= 70`. Query length is a heuristic. Long names can occur for legitimate reasons and must be assessed with context.

Overlap: The three DNS checks overlap. The reviewed register contained 33,778 records matching exactly one check, 4,382 matching exactly two, and 7,761 matching all three. There were 45,921 unique records with at least one DNS check, producing 65,825 finding rows. The 19,904 additional rows are overlapping check matches, not additional unique DNS events.

7.2 Application security

Check	Findings	Severity
`APP_RECORDED_ATTACK_CLASSIFICATION`	79,896	MEDIUM
`APP_SECURITY_ALERT_REVIEW`	79,896	MEDIUM
Total findings	**159,792**	

The two checks have identical coverage in the current dataset: the same 79,896 records match both checks. They use different fields:

1. `APP_RECORDED_ATTACK_CLASSIFICATION`: `app_attack_type` is non-empty.

- `APP_SECURITY_ALERT_REVIEW`: `event_category` equals `SECURITY_ALERT` (case-insensitive).

The two rows for a matching event are separate check findings, not two independent attack events.

Recorded attack-type counts:

Attack type	Records
BRUTE_FORCE	34,599
SCANNER	10,427
SQL_INJECTION	6,896
SSTI	3,579
XSS	3,539
BOT_ACTIVITY	3,512
COMMAND_INJECTION	3,508
SSRF	3,506
AUTH_FAILURE	3,489
LFI	3,445
PATH_TRAVERSAL	3,396
Total	**79,896**

Interpretation guidance:

- **BRUTE_FORCE:** Login failures, invalid credentials, and repeated authentication activity can support a brute-force hypothesis. They do not prove account compromise. HTTP 429 may indicate rate limiting.
- **SQL_INJECTION:** Payloads such as SQL tautologies or `UNION SELECT` indicate probes or attempts. They do not prove that a database query executed or that data was extracted.
- **XSS:** A script payload indicates an attempt. It does not prove that a browser executed the payload; response and browser-side evidence would be needed.
- **COMMAND_INJECTION:** A shell-style payload such as `;id` indicates an attempt. It does not prove command execution.
- **SSRF:** A request targeting a cloud metadata address indicates an SSRF-style attempt. It does not prove an outbound request succeeded or metadata was exposed.
- **SSTI:** A template expression indicates a probe. It does not prove that the template engine evaluated it.
- **LFI and PATH_TRAVERSAL:** File-path payloads indicate attempts to access paths. They do not prove that a file was read or disclosed. These labels describe related but distinct input patterns.
- **SCANNER:** Requests for common administrative or sensitive paths and scanner-like user agents are consistent with automated scanning. User-agent strings can be spoofed.
- **BOT_ACTIVITY:** Automated clients and requests to paths such as `/robots.txt` may be benign or malicious. Automation alone is not proof of malicious intent.
- **AUTH_FAILURE:** Some records labelled as authentication failures did not contain explicit failed-authentication fields. A status code by itself does not establish an authentication failure.

The samples reviewed support treating these records as investigation leads, not confirmed successful exploitation.

7.3 Firewall

Check	Findings	Severity
`FIREWALL_PROTOCOL_FLAG_REVIEW`	212,903	LOW
`FIREWALL_ALLOW_DENY_RULE_REVIEW`	62,208	MEDIUM
Total	**275,111**	

Protocol/flag review: Flags records where the protocol is not TCP but the TCP-flags field is populated. Reviewed examples included ICMP or UDP records with values such as SYN, FIN, ACK, PSH, or URG. This is a broad consistency review. It may reflect source schema conventions or logging semantics and is not, by itself, evidence of malicious traffic.

Allow/deny rule review: Flags records where the action is `ALLOW` while the rule name contains `DENY`. The reviewed records were grouped under `ALLOW` with `DENY-DEFAULT` as the rule value. This may indicate a naming or semantics mismatch. It does not establish that the firewall improperly allowed traffic. The MEDIUM severity is a triage label, not a verdict.

7.4 Linux syslog

Check	Findings	Severity
`LINUX_AUTHENTICATION_FAILURE_REVIEW`	111,462	LOW
`LINUX_PRIVILEGE_ACTIVITY_REVIEW`	106,540	LOW
Total	**218,002**	

The authentication check reviews failed authentication activity. The privilege check reviews sudo/privilege-related activity.

Interpretation cautions:

1 A failed authentication event is not proof of a successful compromise.

1 SSH failed-password and PAM authentication-failure messages can both describe the same authentication attempt. They should not automatically be counted as separate attempts or incidents without a grouping strategy.

1 The current Linux source timestamps lack a year, so `event_timestamp` is NULL. Use the original timestamp text and document the limitation in any timeline.

1 Linux privilege activity is a review lead and requires context about the user, command, host, and expected administrative activity.

7.5 Windows Security

The Windows validator generated **48,942 findings**, which were imported into the unified register:

Check	Findings	Severity
`PROCESS_COMMAND_LINE_REVIEW`	33,261	LOW
`EVENT_ID_CONTEXT_REVIEW`	10,493	INFO
`EVENT_ID_CONTEXT_REVIEW`	5,188	MEDIUM
Total	**48,942**	

The event-ID context findings include Windows events such as:

1 Event ID 1102: audit log cleared.

1 Event ID 4697: service installed.

1 Event ID 4698: scheduled task created.

These events deserve context-based review. Their presence does not independently prove malicious activity.

The process/command-line review identified cases where the normalized process name and command-line executable basename differed. A reviewed example was present in the original source log. The reason for the mismatch was not conclusively established. The finding is retained as a LOW-priority review lead rather than being labelled malicious or a confirmed ingestion defect.

The normalized Windows table contains 311,810 rows. In the reviewed database, `raw_event` and `source_line_number` were populated for all rows, while `source_event_id` was not populated. The separate Windows findings CSV does not contain the original raw event text; consequently, the Windows entries in the unified register have blank raw-event values. The underlying normalized table remains the place to consult for the retained raw event and source-line context.

8. Investigation Register and Reporting

8.1 Register generation

Run from the project root:

```
py -m src.threat_hunt_engine.investigation_register
```

The register generator:

1. Creates the reports directory if it does not exist.
2. Loads existing analyst updates when present.
3. Opens the DuckDB database in read-only mode.
4. Runs source-specific checks for DNS, application security, firewall, and Linux.
5. Imports Windows findings from `windows_validation_findings.csv`.
6. Deduplicates register entries by `investigation_id`, preserving the first instance.
7. Preserves analyst updates across reruns.
8. Writes the unified CSV and summary text output.

Windows validation is a separate upstream step. Run the Windows validator before rebuilding the register if the Windows findings CSV needs to be refreshed.

8.2 Output files

Output	Purpose
`reports/investigation_register.csv`	Structured finding-level investigation register
`reports/investigation_register_summary.txt`	Compact totals and source/severity summary
`reports/archive/investigation_report_*.txt`	Archived timestamped investigation report

The current register contains 767,672 rows and 19 columns. Its size is approximately 418 MiB. The current file is a generated working artifact and is not suitable for ordinary Git tracking.

8.3 Register fields and analyst workflow

The register includes finding identity, source, check name, severity, reason for flagging, investigation status, investigation outcome, and analyst update fields.

At generation time, the reviewed register had:

1. 767,672 unique investigation IDs.
1. 0 blank investigation IDs.
1. 0 duplicate investigation IDs.
1. 0 malformed CSV rows.
1. 767,672 records with status `New`.
1. 767,672 records with outcome `Undetermined`.
1. Blank `analyst_notes` values, as expected before analyst review.

The status and outcome values are workflow defaults, not findings about the events themselves. Analysts should update records only after examining supporting evidence.

8.4 Current severity totals

Severity	Findings
INFO	10,493
LOW	472,060
MEDIUM	285,119
Total	**767,672**

Severity indicates review priority assigned by the checks. It is not a probability of compromise and does not replace analyst validation.

9. Reproducing the Workflow

9.1 Prepare the project

1. Obtain the project source code.
2. Obtain the five source datasets from their authorized source or prepare compatible test inputs.
3. Place the expected input files under `logs/` using the filenames expected by the ingestion script.
4. Install Python and the dependency in `requirements.txt`.
5. Run the commands from the project root.

9.2 Recommended execution sequence

```
# 1. Check Python and dependency
py --version
py -m pip install -r requirements.txt

# 2. Review source files and metadata
py -m src.threat_hunt_engine.source_metadata_review

# 3. Review source quality
py -m src.threat_hunt_engine.source_quality_review

# 4. Ingest all five sources
py -m src.threat_hunt_engine.ingestion

# 5. Validate database and identifiers
py -m src.threat_hunt_engine.database_validation
py -m src.threat_hunt_engine.duplicate_id_check

# 6. Generate Windows validation findings
py -m src.threat_hunt_engine.validate_windows_events

# 7. Generate the unified investigation register
py -m src.threat_hunt_engine.investigation_register

# 8. Generate the timestamped investigation report
py -m src.threat_hunt_engine.investigation_checks
```

The exact order can vary for isolated validation work. For a complete fresh run, generate the normalized database before database-dependent checks, generate the Windows findings CSV before the unified register, and retain the command output for reproducibility.

9.3 Expected current-dataset checkpoints

For the reviewed dataset and code state, the expected normalized row counts are:

1. DNS: 350,000
1. Application security: 438,739
1. Firewall: 394,170
1. Linux syslog: 977,509
1. Windows Security: 311,810

The current unified register checkpoint is 767,672 findings. Different datasets, parser revisions, thresholds, or source-file versions will produce different counts. These values are reference checkpoints, not universal expected results for arbitrary inputs.

9.4 Reproducibility notes

1. Run commands from the repository root so module imports and project-relative paths resolve correctly.
1. Preserve the original source files and record their provenance.
1. Record Python and DuckDB versions.
1. Keep a copy of generated outputs for each meaningful run if auditability is required.
1. Do not edit raw source files in place during a test run.
1. If ingestion uses `CREATE OR REPLACE TABLE`, a full run replaces the existing normalized tables.

1. Use the Linux-only option only when intentionally rebuilding the Linux table while preserving the other tables.

10. Limitations, Assumptions, and Safe Interpretation

10.1 No cross-source correlation

The current investigation register is source-by-source. It does not correlate a DNS event, firewall event, application event, Linux event, and Windows event into a single attack chain. Findings from different sources must not be assumed to describe the same activity without additional correlation logic and evidence.

10.2 Heuristic checks

The checks use source labels, field values, thresholds, event IDs, and pattern matching. These are triage mechanisms. They can generate false positives, duplicate review leads, or miss activity outside the implemented conditions.

10.3 Findings are not incidents

A finding means that a record met a check condition. It does not prove:

1. that an attacker controlled the source IP;
1. that an exploit succeeded;
1. that a system was compromised;
1. that data was accessed or exfiltrated;
1. that a firewall rule was incorrectly applied;
1. that an authentication attempt resulted in account access.

Analyst conclusions should be based on corroborating evidence, relevant baselines, asset and identity context, and the original event data.

10.4 Timestamp handling

Linux source timestamps lack a year and are not currently converted into populated `event_timestamp` values. This affects time-based analysis and cross-source sequencing.

10.5 Windows raw-event limitation in the register

Windows findings are imported from a validation CSV that does not include `raw_event`. As a result, raw event text is blank for Windows findings in the unified register. The normalized Windows table retains raw event text and source line numbers for lookup.

10.6 Dataset and publication restrictions

The project code and the input datasets are separate intellectual-property and redistribution questions. A code licence such as MIT does not automatically grant permission to redistribute datasets, logs, or third-party samples.

Before publishing the repository:

1. Confirm the origin and licence of every dataset.
1. Check logs for personal data, credentials, tokens, internal hostnames, private IPs, email addresses, and other sensitive details.
1. Do not publish the full DuckDB database or large generated reports by default.
1. Do not commit large raw datasets unless their licence explicitly permits redistribution and the repository hosting limits are appropriate.
1. Prefer a small, synthetic or explicitly redistributable sample dataset for clone-and-test instructions.
1. Exclude generated databases, large CSV reports, caches, and local environment files from Git unless there is a documented reason to include them.

10.7 Operational safety

Run the project on data you are authorized to access. Treat outputs as investigative leads. Protect source logs and generated findings according to the sensitivity of the underlying environment.

Appendix A. Investigation Check Reference

Source	Check	Current findings	Main review purpose
DNS	`DNS_SOURCE_LABEL_REVIEW`	45,921	Review source-provided DNS labels
DNS	`DNS_HIGH_ENTROPY_QUERY_REVIEW`	12,010	Review high-entropy subdomains
DNS	`DNS_LONG_QUERY_REVIEW`	7,894	Review long DNS query names
Application	`APP_RECORDED_ATTACK_CLASSIFICATION`	79,896	Review non-empty recorded attack types
Application	`APP_SECURITY_ALERT_REVIEW`	79,896	Review application security alert category
Firewall	`FIREWALL_PROTOCOL_FLAG_REVIEW`	212,903	Review protocol/TCP-flag combinations
Firewall	`FIREWALL_ALLOW_DENY_RULE_REVIEW`	62,208	Review ALLOW action with DENY-named rule
Linux	`LINUX_AUTHENTICATION_FAILURE_REVIEW`	111,462	Review failed authentication activity
Linux	`LINUX_PRIVILEGE_ACTIVITY_REVIEW`	106,540	Review privilege-related activity
Windows	`PROCESS_COMMAND_LINE_REVIEW`	33,261	Review process/command-line differences
Windows	`EVENT_ID_CONTEXT_REVIEW`	15,681	Review selected Windows event IDs

The Windows `EVENT_ID_CONTEXT_REVIEW` total is the sum of 10,493 INFO and 5,188 MEDIUM findings.

Appendix B. Engineering Decisions and Observed Issues

1. **DuckDB storage:** A local analytical database supports SQL-based inspection over large event collections without requiring a separate database server.
1. **Source-specific normalized tables:** Separate tables preserve source-specific fields while providing a common event structure.
1. **Read-only investigation queries:** Investigation and validation workflows use read-only database connections where appropriate, reducing the risk of modifying normalized data during analysis.
1. **Separate Windows validation:** Windows-specific findings are written to a CSV and imported by the unified register, rather than being generated by the same SQL checks used for the other four sources.
1. **Finding-level deduplication:** The unified register deduplicates by `investigation_id`; this is not a claim that source events are globally unique or that cross-source duplicate activity has been resolved.
1. **Preservation of analyst updates:** Existing analyst updates are loaded before the register is rewritten so that rerunning the workflow does not automatically discard those updates.
1. **Linux timestamp limitation:** Original timestamp text is retained because the source format does not provide a year. The normalized timestamp remains NULL.
1. **Windows process-name mismatch:** The observed mismatch remains a review lead because its cause was not conclusively established.
1. **Application check overlap:** The two application checks produce the same 79,896-record coverage in the reviewed dataset. Their combined 159,792 findings are two check matches per record, not twice as many unique application events.
1. **DNS check overlap:** Multiple DNS checks can match one event. Finding totals must be distinguished from unique event totals.

1. **Firewall semantics:** `ALLOW` paired with `DENY-DEFAULT` may reflect a naming or interpretation issue; it is not sufficient evidence of an enforcement failure.

Appendix C. Evidence and Screenshot Checklist

Capture evidence from the actual environment when preparing a public project demonstration or assessment submission. Do not fabricate screenshots or expose secrets.

Suggested evidence:

1. Project directory structure.
2. Python version and installed DuckDB version.
3. Successful full ingestion command and per-table row counts.
4. Database validation output.
5. Duplicate identifier review output.
6. Source metadata and source-quality review output.
7. Windows validation command and findings count.
8. Investigation register generation output.
9. Investigation register summary showing source and severity totals.
10. Example finding rows with sensitive values redacted.
11. A DuckDB schema or table-inspection view.
12. A demonstration of analyst status/notes persistence, if included in the final public demo.

Use clear filenames, for example:

```
01-project-structure.png
02-python-duckdb-versions.png
03-full-ingestion-results.png
04-database-validation.png
05-duplicate-id-review.png
06-source-quality-review.png
07-windows-validation.png
08-investigation-register-generation.png
09-register-summary.png
10-sample-findings-redacted.png
```

Store screenshots in a dedicated documentation evidence folder only after reviewing them for credentials, personal information, internal addresses, and other sensitive details.

Document status

This document describes the current observed implementation and dataset results. Re-run validation after code, data, dependency, or schema changes. Update the counts and limitations when the implementation changes.